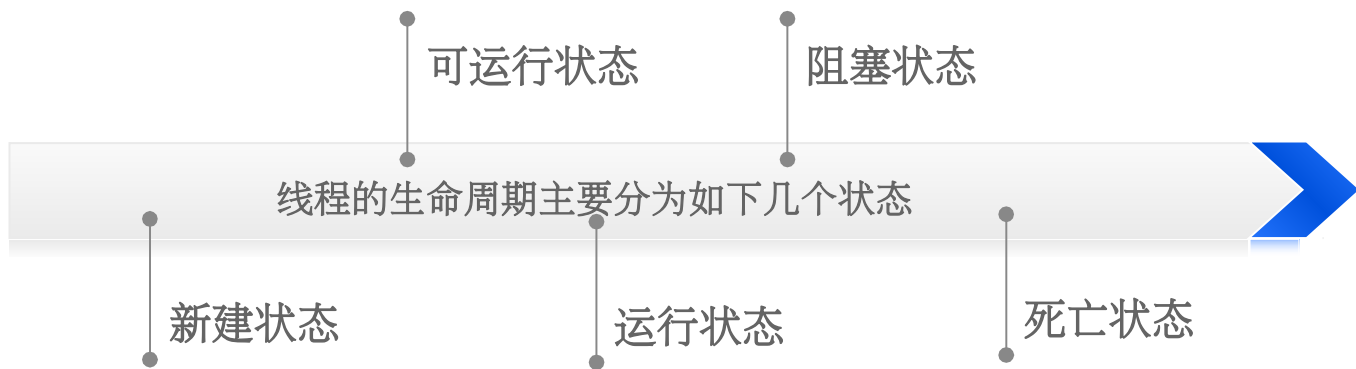


2 线程的状态与生命周期

- 线程创建后，就开始了它的生命周期，在不同的生命周期阶段线程有不同的状态。对线程调用各种控制方法，就使线程从一种状态转换为另一种状态。



2.1 新建状态（New）

新建状态

调用一个线程类的构造方法，便创建了一个线程，如：

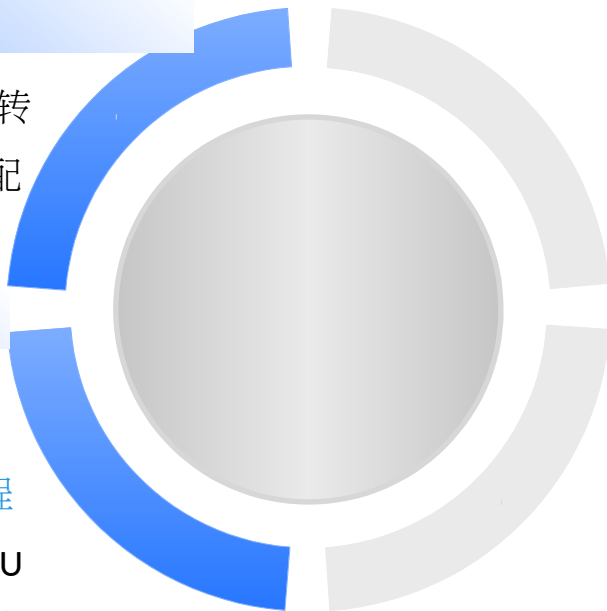
```
Thread t1=new Thread();
```

此时线程处于新建状态。处于新建状态的线程还没有被分配有关的系统资源。

2.2 可运行状态（Runnable）

处于新建状态的线程调用 `start()`，使线程的状态转换为可运行状态。`start()` 方法使系统为线程分配必要的资源，并将线程交给系统调度。

此时线程仅仅是可以运行，但不一定在运行中。在多线程程序设计中，系统中往往会有多个线程同时处于 Runnable 状态，它们将竞争有限的 CPU 资源，由 JVM 根据线程调度策略进行调度。



2.3 运行状态（Running）

运行状态是线程占有 CPU 并实际运行的状态。此时线程状态的变迁有如下三种情况：

如果线程正常执行结束或应用程序停止运行，线程将进入死亡状态。

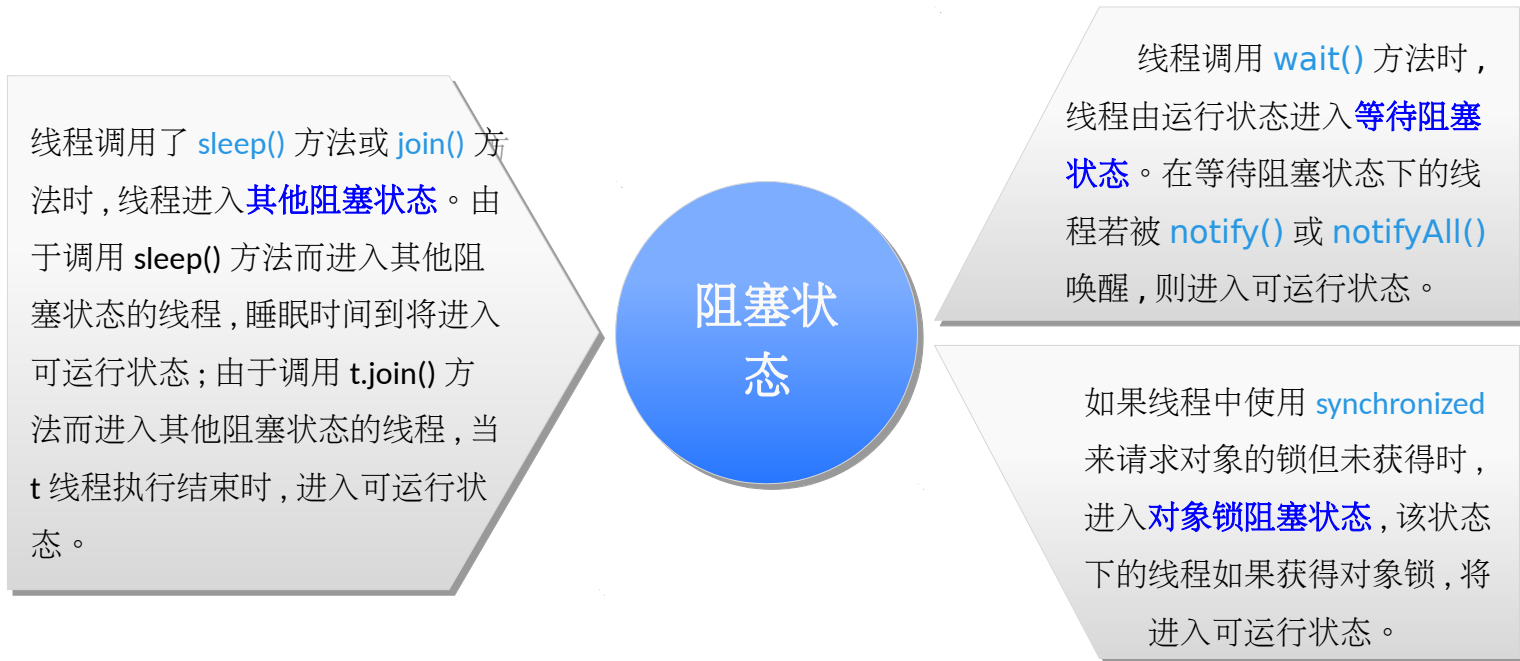
如果当前线程执行了 `yield()` 方法，或者当前线程因调度策略（执行过程中，有一个更高优先级的线程进入可运行状态，这个线程立即被调度执行，当前线程占有的 CPU 被抢占；或在分时方式时，当前执行线程执行完时间片）由系统控制进入可运行状态。

如果发生下面几种情况时，线程就进入阻塞状态

- 线程调用了 `sleep()` 方法或 `join()` 方法，进入阻塞状态
- 线程调用 `wait()` 方法时，由运行状态进入阻塞状态
- 如果线程中使用 `synchronized` 来请求对象锁但未获得时，进入阻塞状态

2.4 阻塞状态（Blocked）

阻塞状态根据产生的原因又可分为对象锁阻塞、等待阻塞和其他阻塞。状态相应变迁如下：



2.5 死亡状态 (Dead)

一个线程对象在以下三种情况下，会结束并且进入死亡状态：

自然撤消：是指从线程的
`run()` 方法执行完毕后正常退出。

1

强制退出：调用线程的实例
方法 `stop()` 可以强制停止当前线程。由于 `stop()` 方法会导致死锁，目前该方法已过时，不推荐使用。

2

线程执行时遇到一个未捕获的异常，线程被终止并且进入死亡状态。

3

线程生命周期中的状态变迁示意图

